SPOTY

Estacione com inteligência

Backlog & Relatório
do Projeto

Estado atual, funcionalidades entregues, problemas encontrados
e próximas prioridades de desenvolvimento

✓ Entregue	PARCIAL	PENDENTE	PROBLEMA

Versão atualizada • 22 de outubro de 2026

Documento interno de planejamento — baseado no BACKLOG.md e no estado do código descrito no documento.

Sumário

1. Onde o projeto está
2. Problemas encontrados
3. Backlog priorizado
4. Decisões em aberto

1. Onde o projeto está

Site estático (HTML + CSS + JS puro, sem build) com autenticação via Supabase e mapa real. Ainda não existe nenhuma funcionalidade de vagas: o mapa mostra só a localização buscada, sem estacionamentos.

✓ Entregue

- Landing page: hero, busca, depoimentos, "como funciona", rodapé
- Mapa Leaflet + CARTO limitado ao território brasileiro
- Busca por endereço (Nominatim) e "Usar minha localização" (geolocalização do navegador)
- Cadastro (nome, usuário, e-mail, telefone, senha) → Supabase Auth + tabela `profiles`
- Login, logout, menu do usuário com iniciais no cabeçalho (`sessao.js`)
- Editar informações (nome, usuário, e-mail, telefone, senha)
- Páginas institucionais: Para empresas, Central de ajuda, Fale conosco, Termos de uso, Privacidade
- Identidade visual com variáveis CSS, transição entre páginas, toasts
- Licença MIT

PARCIAL Parcial / só aparência

- Vagas próximas: o card do mapa tem esse título, mas não há vagas nem marcadores
- Fale conosco: valida o formulário e só escreve no `console.log`; nada é enviado
- Para empresas: só texto e um botão para o Fale conosco; não há cadastro de estabelecimento
- Textos de marketing sem funcionalidade por trás: "vagas em tempo real", "pagamento digital", PIX/saldo Spoty, "milhares de motoristas", depoimentos (fictícios)
- Selos App Store / Google Play apontam para a página inicial das lojas

ATUALIZAÇÃO O que mudou em relação ao backlog v1.0 (setembro)

Item no v1.0	Situação hoje (código)
Login/autenticação: Pendente	✓ Funciona (Supabase Auth); a sessão persiste
Cadastro "não persiste em lugar nenhum"	✓ Grava em Supabase Auth + `profiles`
Editar informações	✓ Novo, não estava no v1.0
Vagas e marcadores simulados (aleatórios) no mapa	ATENÇÃO Não existem mais no código: o mapa só mostra o ponto buscado
Banco: MySQL + Workbench, schema modelado (usuarios, estabelecimentos, vagas, reservas, pagamentos, assinaturas)	ATENÇÃO Não está em uso. O projeto usa Supabase/Postgres, que o v1.0 tinha descartado; nenhuma tabela do schema MySQL é usada
Back-end em Python: Pendente	Não existe; hoje o Supabase faz o papel de back-end
Cadastro em duas vertentes	PENDENTE Segue pendente (formulário único)
Painel do estabelecimento, pagamentos, app mobile	PENDENTE Seguem pendentes

Item no v1.0	Situação hoje (código)
Termos e Privacidade (rascunho, validação final)	PARCIAL Segue igual
Selos das lojas (arte pronta, app inexistente)	PARCIAL Segue igual
Fale conosco (só front)	PARCIAL Segue igual
Detecção real de vagas: "não conseguimos"	PENDENTE Segue sem solução (ver P3)

2. Problemas encontrados PROBLEMA

#	Problema	Onde	Sugestão
B1	`html/index.html` é cópia idêntica do `index.html` da raiz, mas com caminhos `./css`, `./scripts` que quebram dentro de `/html` (CSS dá 404)	`html/index.html`	Apagar o arquivo
B2	`conectado.html` / `conectado.js` não são linkados por nada; o login redireciona para `../index.html`	`html/conectado.html`	Remover ou integrar (ex.: virar a "minha conta")
B3	O schema do banco (`profiles`) e as políticas RLS não estão no repositório; quem clonar não consegue subir o projeto	repo	Versionar `supabase/schema.sql`
B4	O cadastro faz `insert` em `profiles` logo após o `signUp`. Com "Confirm email" ligado no Supabase não há sessão ainda, e a política RLS bloqueia o insert (conta criada sem perfil)	`cadastro.js`	Criar o perfil por trigger no banco (`auth.users` → `profiles`)
B5	URL e chave anon do Supabase fixas em `supabaseClient.js`. A chave anon é pública por design, mas a segurança depende 100% do RLS, que não está versionado	`supabaseClient.js`	Documentar, versionar RLS, criar `supabaseClient.example.js`
B6	Funções repetidas em vários arquivos: `mostrarToast`, `transicaoPagina`, `pegarIniciais`, `alternarVisibilidadeSenha`, `deixarIniciaisMaiusculas`	`scripts/*.js`	Extrair para `scripts/comum.js`
B7	Só o `index.html` carrega `sessao.js`; as páginas institucionais sempre mostram "Cadastre-se", mesmo com usuário logado	`html/empresas.html` etc.	Cabeçalho único com estado de sessão
B8	Nominatim (OpenStreetMap) tem política de uso restrita (≈1 req/s, uso leve); não serve para produção	`script.js`	Geocoder próprio/pago (Mapbox, Google, LocationIQ) quando houver tráfego
B9	Sem `.gitignore`, sem README (foi apagado no último commit), sem testes, sem deploy	repo	README entregue junto com este backlog; ver itens P0
B10	Validação fraca: senha mínima de 6 caracteres, telefone sem verificação, nome de usuário sem normalização (caixa/espaços)	`cadastro.js`	Regras de senha, `usuario` em minúsculas sem espaços

3. Backlog priorizado

P0 — Fundação (antes de construir mais coisa)

- ☐ Publicar o README (entregue junto) na raiz do repo
- ☐ `supabase/schema.sql` com tabela `profiles` + RLS + trigger de criação de perfil (B3, B4)
- ☐ Apagar `html/index.html` (B1) e decidir o destino de `conectado.html` (B2)
- ☐ `.gitignore` + `scripts/supabaseClient.example.js` (B5)
- ☐ Extrair código repetido para `scripts/comum.js` (B6)
- ☐ Deploy (GitHub Pages, Netlify ou Vercel) e configurar Site URL / Redirect URLs no Supabase

P1 — Núcleo do produto: achar vaga

- ☐ Modelo de dados: reaproveitar o schema do v1.0 (`estabelecimentos`, `vagas`, `reservas`, `pagamentos`, `assinaturas`), traduzido para o banco escolhido (ver decisão 1). No Supabase: ids `uuid`, `dono_id` apontando para `auth.users`, RLS em todas as tabelas
- ☐ Habilitar PostGIS no Supabase para busca por raio (ou índice espacial, se for MySQL)
- ☐ Dados de exemplo (seed) de estacionamentos, marcados como simulados
- ☐ Marcadores no mapa + lista com distância, preço e status
- ☐ Filtros (preço, distância, aberto agora)
- ☐ Página/painel de detalhe do estacionamento + botão "Como chegar"
- ☐ Rótulo honesto: sem sensores/satélite não há detecção real de vaga livre em via pública; trocar "tempo real" por "informado pelo estabelecimento" enquanto isso

P1 — Contas e perfis

- ☐ Conta com papéis múltiplos: uma mesma conta pode ser dono de estacionamento e cliente (não é um tipo fixo e exclusivo). Modelar como relação (`estabelecimentos.owner_id`), não como campo `tipo` em `profiles`
- ☐ Cadastro com dois caminhos: "Sou dono/estabelecimento" e "Quero estacionar"
- ☐ Recuperar senha (fluxo "esqueci minha senha")
- ☐ Login com Google (opcional)

P2 — Lado do dono (monetização)

- ☐ Painel do estabelecimento: nome, total de vagas, vagas ocupadas/livres, preço por 30 min / 1 hora / diária, horário de funcionamento e controle de entrada/saída por placa (atualização manual no início; já especificado no v1.0)
- ☐ Cadastro real de estabelecimento na página "Para empresas"
- ☐ Assinatura mensal do painel (Mercado Pago / Stripe / Pix recorrente)
- ☐ Estabelecimentos assinantes recomendados a usuários próximos

P2 — Lado do motorista

- ☐ Favoritos e histórico de buscas
- ☐ Avaliações de estacionamentos
- ☐ Reserva de vaga + pagamento (Pix/cartão)
- ☐ Depoimentos reais (substituir os fictícios) ou remover a seção até existirem

P2 — Qualidade

- ☐ Fale conosco funcional (tabela `contatos` + e-mail via Edge Function)
- ☐ Revisão de acessibilidade (labels, contraste, foco) e SEO básico (meta description, Open Graph)
- ☐ Testes mínimos (fluxo de cadastro/login) e checklist de RLS
- ☐ Revisar Termos e Privacidade (LGPD: coleta de telefone e localização) antes de abrir ao público

P3 — Futuro

- ☐ Transformar em PWA (instalável no celular) antes de investir em app nativo
- ☐ App nativo (Android/iOS) e links reais nas lojas
- ☐ Anúncios opcionais para o usuário final (recompensa pelo uso)
- ☐ Detecção real de vagas (hoje não há sensores nem satélites). Caminhos do v1.0, do mais viável ao mais caro: (1) donos informam a ocupação pelo painel; (2) sensores IoT em estacionamentos parceiros; (3) câmeras com reconhecimento de imagem; (4) dados públicos de mobilidade urbana, quando existirem. Opcional: atualização colaborativa ("tem vaga aqui")
- ☐ Geocoder e tiles de produção (B8)

4. Decisões em aberto (para conversarmos)

1. Banco e back-end (decisão que mudou): o v1.0 fechou MySQL + Workbench + back-end em Python (e descartou Supabase), mas o código no GitHub já usa Supabase (Auth + Postgres direto do navegador, sem back-end próprio). Os dois caminhos:

- A) Ficar no Supabase: mais rápido (login, hash de senha, RLS e API prontos). Custo: traduzir o schema do v1.0 para Postgres e aprender RLS.
- B) Voltar para MySQL + Python: mais controle e mais aprendizado de back-end (API em Flask/FastAPI, bcrypt, JWT). Custo: reescrever login/cadastro/edição (`cadastro.js`, `login.js`, `sessao.js`, `editar-informacoes.js`, `conectado.js`) e hospedar a API.

Seja qual for a escolha, o README e o backlog se ajustam a ela.

2. Vagas simuladas: foram removidas do código. Queremos recolocar marcadores de exemplo (rotulados como simulados) como primeiro passo do P1?

3. Escopo do MVP: mostrar estacionamentos cadastrados (comerciais) primeiro e deixar vagas de rua para depois?

4. Região inicial: lançar só em uma região (ex.: Zona Leste de SP / ABC) simplifica o cadastro inicial de estabelecimentos.